

Intégration du Système

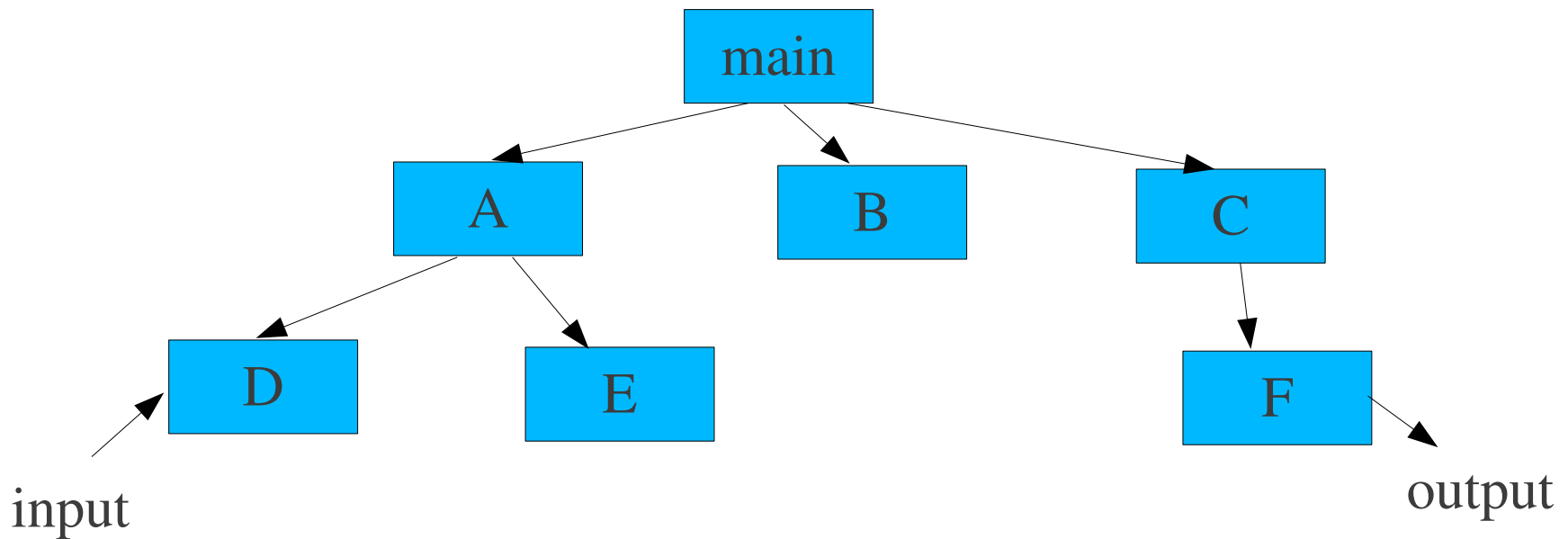
Lectures: Daniel Galin chap 9.2, Robert Binder Chap 13

Stratégie d'intégration du Système

- Détermine comment des modules de bas-niveau sont assemblés pour former des modules de plus haut-niveau
- Impact sur
 - la forme d'écriture des tests unitaires
 - le type d'outils utilisés
 - l'ordre de codage/test des unités
 - le coût de génération des cas de tests
 - le coût de localisation/correction des erreurs détectées

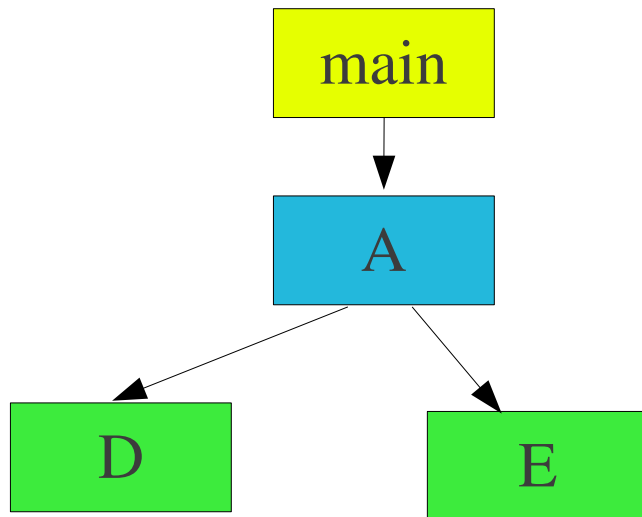
Tests d'intégration

- Objectif: assurer que des modules fonctionnant correctement en isolation, fonctionnent mis ensemble
- Nécessite une représentation de *structure d'invocation de modules*



Tests d'intégration

- Comment assurer qu'un composant est testé de façon effective en isolation?



En supposant que tous les composants fonctionnent correctement en isolation, quels types de fautes peut-on s'attendre à découvrir?

Tests d'intégration

Stubs

- Stub – remplace un module invoqué
 - Stub pour module de lecture passe données de tests
 - Stub pour module de *sortie* retourne les résultats de tests
- Peut remplacer des composants
 - ex: base-de-donnée, réseau
- Doit être déclaré/invoqué comme le module *réel*
 - même nom que module remplacé
 - même liste de paramètres
 - même type de retour
 - même modificateurs (static, public ou private)

Tests d'intégration

Stubs

```
public static int someFunc(float fThat)
```

Exemple de Stub

```
public static int someFunc(float fThat) {  
    int iDummy = 42;  
    System.out.println (“In method  someFunc ” +  
        “Input float =” + fThat);  
    System.out.println (“Returning 42.”);  
    return iDummy;  
}
```

Tests d'intégration

Stubs

- Fonctions courantes d'un stub
 - afficher/enregistrer message de trace
 - afficher/enregistrer paramètres passés
 - retourner valeur selon objectif de test – peut être
 - à partir d'une table
 - d'un fichier externe
 - basé sur une recherche selon la valeur de paramètres
 - ...

Tests d'intégration

Stubs

```
void main() {  
1  int x, y;  
2  x = A();  
3  if (x > 0) {  
4    y = B(x);  
5    C(y);  
    } else {  
6    C(x)  
    }  
7  exit(0);  
}
```

- Test pour 1 – 2 – 3 - 4 – 5 – 7
 - stub pour A() tel $x > 0$ retourné
- Test pour 1 – 2 – 3 – 6 – 7
 - stub pour A() tel que $x \leq 0$ retourné

Problème : les stubs peuvent devenir trop complexes

Tests d'intégration

Drivers

- Driver - module utilisé pour invoquer les modules sous test
 - passage de paramètres
 - traitement de valeurs retournées

```
public class TestSomething {  
    public static void main (String[] args) {  
        int iReturn = 0;  
        float iSend = 98.6f;  
        Whatever what = new Whatever;  
        System.out.println (“Sending someFunc 98.6.”);  
        iReturn = what.someFunc (iSend);  
        // or iReturn = what.someFunc(98.6);  
        System.out.println (“SomeFunc returned: ” + iReturn);  
    }  
}
```

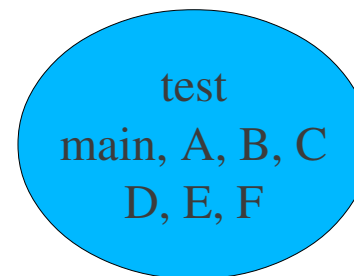
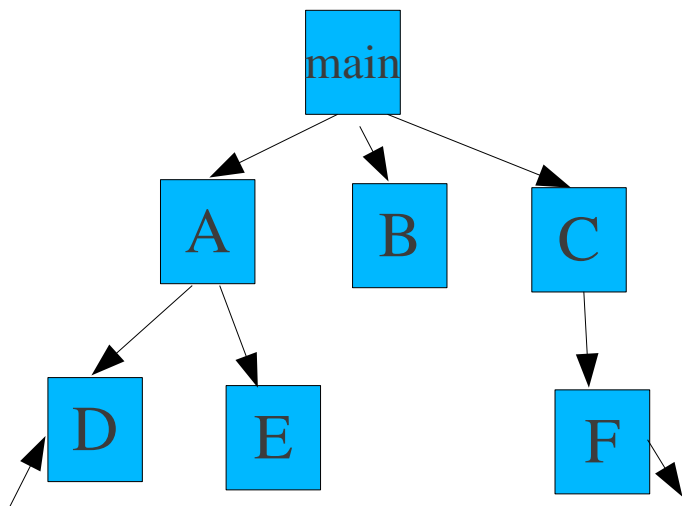
Stratégies de tests d'intégration

- Différentes stratégies pour l'intégration
- Critères de Comparaison
 - localisation de fautes
 - effort (pour stubs et drivers)
 - degré de test des modules
 - possibilité de développement parallèle

Intégration Big Bang

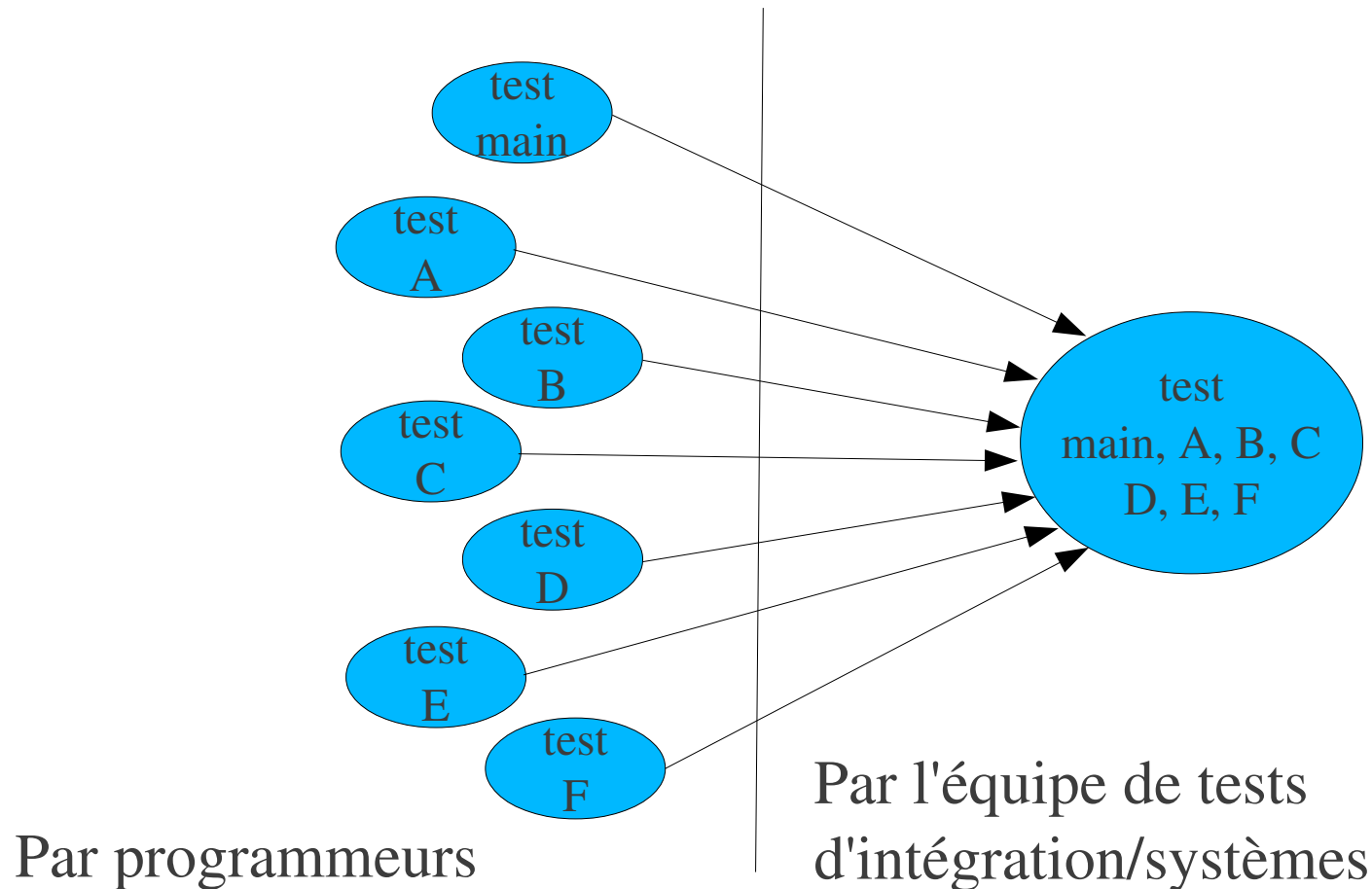
- Stratégie non-incrémentielle

1. Intégration des composants comme un tout



Intégration Big Bang

- Suppose que les composants sont initialement testés en isolation
 - responsabilité des programmeurs

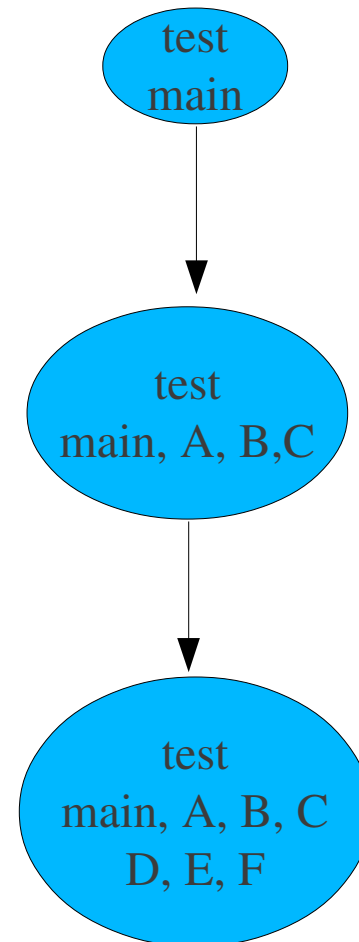
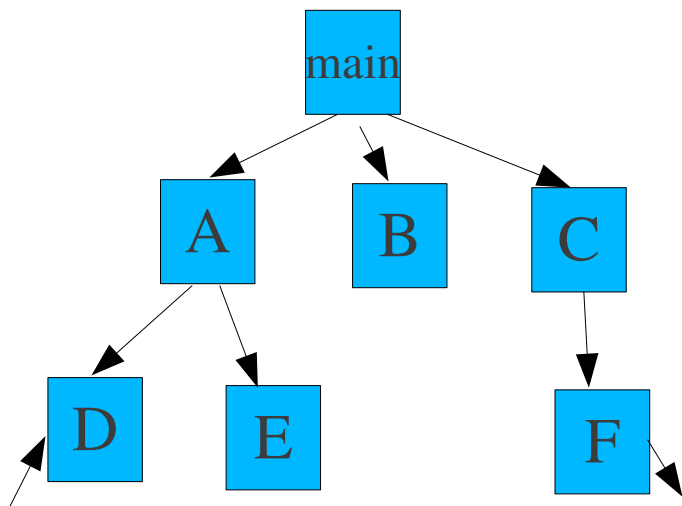


Intégration Big Bang

- Avantages
 - convient aux systèmes petits/stables
- Dés-avantages
 - ne permet pas le développement en parallèle
 - difficile de localiser les fautes
 - facile de rater des fautes d'interface

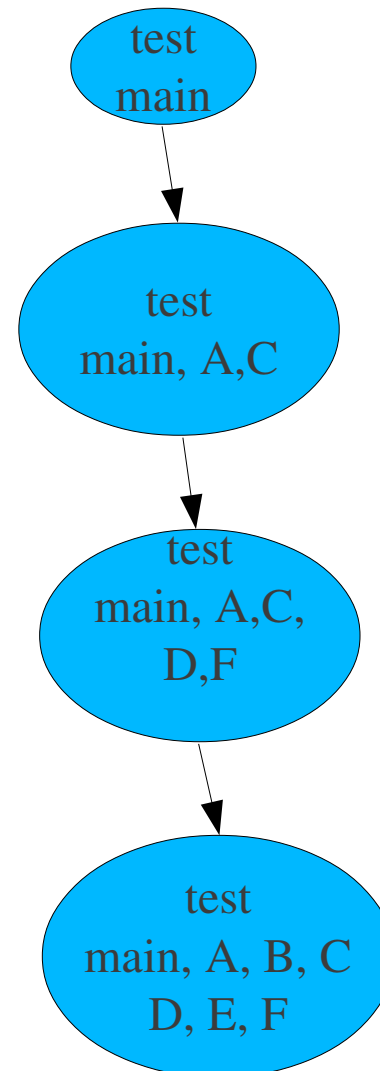
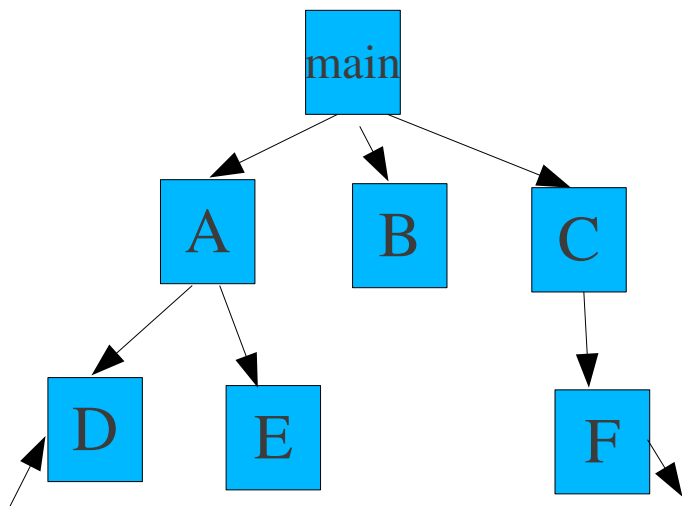
Intégration Top-down

- Stratégie incrémentielle
 1. Tester les modules de haut-niveau, puis
 2. Module appelés jusqu'aux modules de plus bas niveau



Intégration Top-down

- Possible d'altérer l'ordre tel sont testés le plus tôt possible
 - unités critiques
 - unités d entrées/sorties

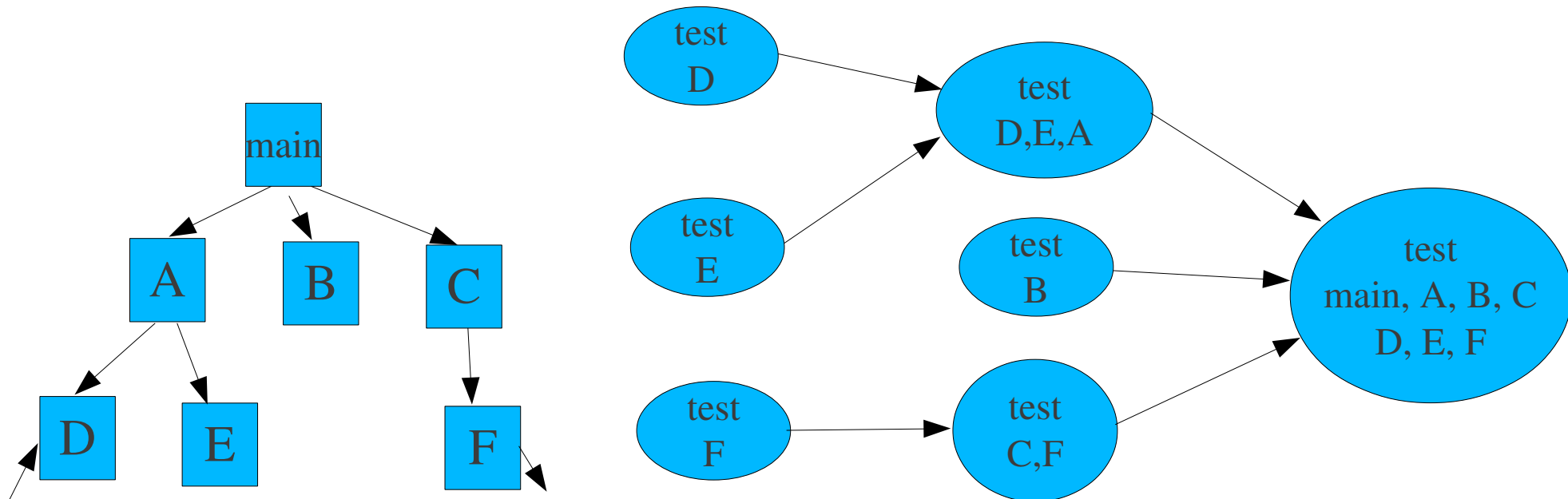


Top-down Intégration

- Avantages
 - Facilité de localisation de fautes
 - Peu ou pas de *drivers*
 - Possibilité d'obtenir un *prototype*
 - Différents ordres de test/implémentation possibles
 - Problèmes majeures de design trouvés en premier
 - dans modules de la logique au dessus de la hiérarchie
- Dés-avantages
 - Nécessite beaucoup de stubs
 - Modules potentiellement réutilisables (en dessous de la hiérarchie) peuvent être testés insuffisamment

Intégration Bottom-up

- Stratégie incrémentielle
 1. Tester les modules de bas-niveau, puis
 2. Modules les appelant jusqu'au module de plus haut-niveau

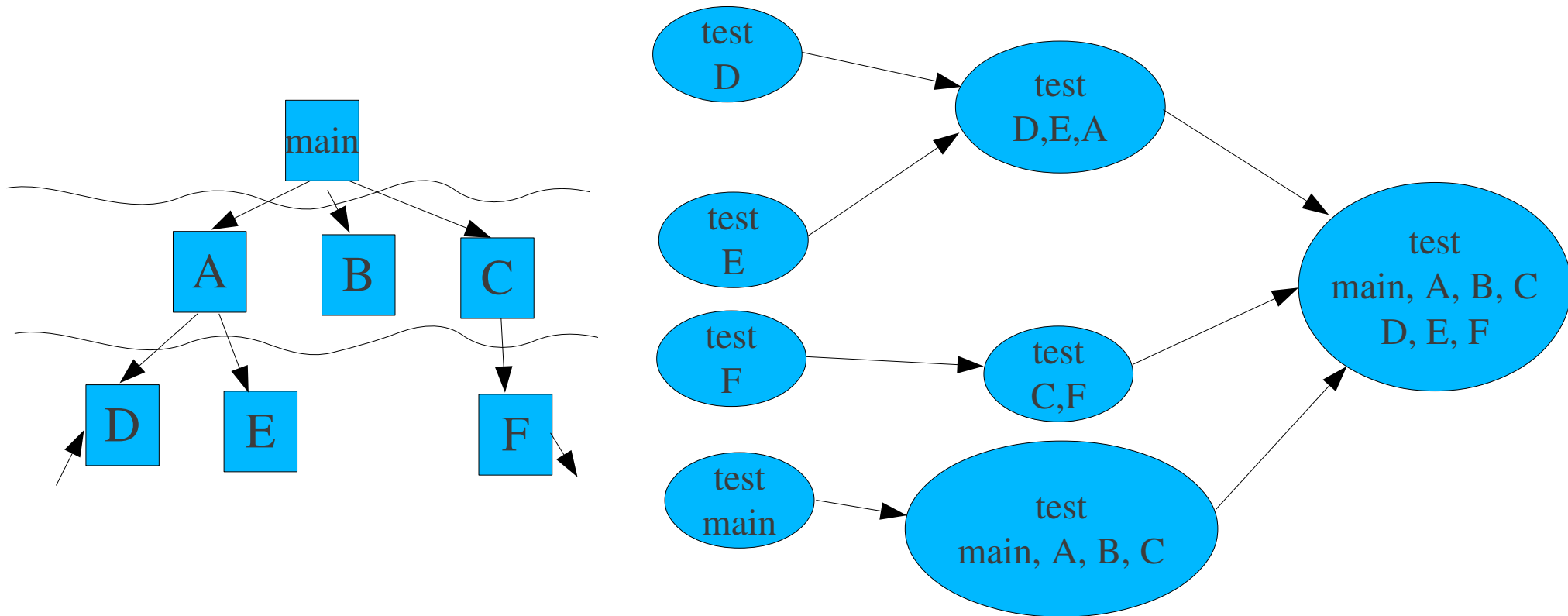


Intégration Bottom-up

- Avantages
 - Localisation de fautes plus effective (que big-bang)
 - Pas besoin de stubs
 - Test en profondeur de modules réutilisables
 - Test peut être en parallèle avec l'implémentation
- Dés-avantages
 - Besoin de *drivers*
 - Modules de haut-niveau (liés à la logique de la solution) testés en dernier et le moins
 - Pas de concept de squelette *prototype*

Intégration Sandwich

- Combine approches top-down et bottom-up
- Distingue 3 couches
 - logique (au dessus) - testée selon top-down
 - intermédiaire
 - opérationnelle (dessous) – testée selon bottom-up



Intégration basée sur le risque

- Intégrer selon la criticalité
 - modules critiques ou complexes ainsi que modules qu'ils invoquent intégrés en premier

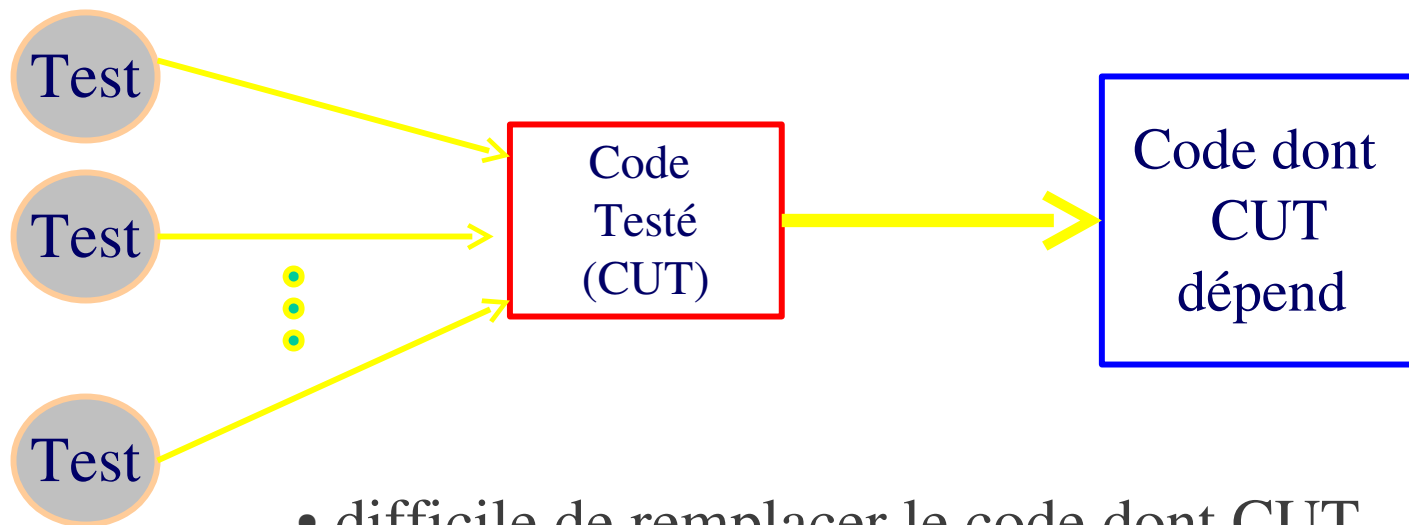
Intégration basée Fonctions/Threads

- Intégration de modules selon les threads/fonctions auquel ils appartiennent

Intégration de systèmes Orientés-Objets

- Différents niveaux
 - Intra Classe (méthodes dans une classe)
 - big-bang
 - bottom-up: constructeurs - accesseurs - predicats - modificateurs - itérateurs – destructeurs
 - intégration basée états
 - Intégration de Cluster (Package)
 - Big Bang
 - Bottom-up
 - Top-down
 - Basée Scénarios
 - Intégration de Sous-système/Systeme
 - Big Bang
 - Bottom-up
 - Top-down
 - Basée Cas d'utilisations

Intégration de systèmes Orientés-Objets – Mock Objects

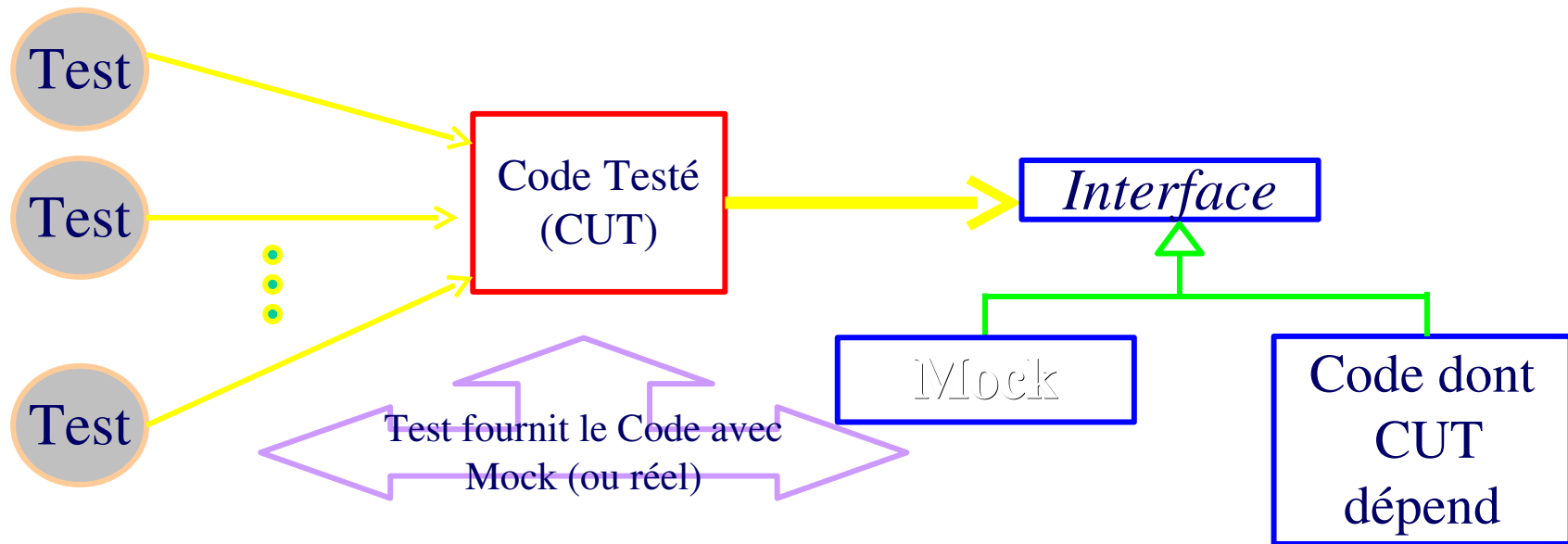


- difficile de remplacer le code dont CUT dépend de façon flexible
 - sans changer CUT
 - maintenir une bibliothèque d'objets stubs

Intégration de systèmes Orientés-Objets

- Mock Object – forme de *stubs*
 - basés sur les interfaces
 - plus facile à initier et contrôler
 - isole le code de détails pouvant être fournis plus tard
 - peut-être incrémentalement raffinés en remplaçant par le code réel

Intégration de systèmes Orientés-Objets – Mock Objects



- Basé sur le principe d'inversion de dépendance
- Tests contrôle le comportement du mock
 - test détermine quoi utiliser
- Mock remplace le code de façon transparente

Exemple: test de
servlet

```
1 public void doGet(HttpServletRequest req,
2                     HttpServletResponse res)
3     throws ServletException, IOException
4 {
5     String str_f = req.getParameter("Fahrenheit");
6
7     res.setContentType("text/html");
8     PrintWriter out = res.getWriter();
9
10    try {
11        int    temp_f = Integer.parseInt(str_f);
12        double temp_c = (temp_f - 32) * 5.0 / 9.0;
13        out.println("Fahrenheit: " + temp_f +
14                    ", Celsius: " + temp_c);
15    }
16    catch (NumberFormatException e) {
17        out.println("Invalid temperature: " + str_f);
18    }
```

Intégration de systèmes Orientés-Objets – Mock Objects

- Comment effectuer le test unitaire sans
 - impliquer un serveur web, container servlet ?
- Comment automatiser les tests ?
 - sans avoir à vérifier manuellement les résultats avec un web browser
 - utiliser quelque-chose comme *junit*

```

1 public void test_bad_parameter() throws Exception {
2     TemperatureServlet s = new TemperatureServlet();
3     MockHttpServletRequest request = new MockHttpServletRequest();
4     MockHttpServletResponse response = new MockHttpServletResponse();
5
6     request.setupAddParameter("Fahrenheit", "boo!");
7     response.setExpectedContentType("text/html");
8     s.doGet(request, response);
9     response.verify();
10    assertEquals("Invalid temperature: boo!\r\n",
11                response.getOutputStreamContents());
12 }
13
14 public void test_boil() throws Exception {
15     TemperatureServlet s = new TemperatureServlet();
16     MockHttpServletRequest request =
17         new MockHttpServletRequest();
18     MockHttpServletResponse response =
19         new MockHttpServletResponse();
20
21     request.setupAddParameter("Fahrenheit", "212");
22     response.setExpectedContentType("text/html");
23     s.doGet(request, response);
24     response.verify();
25     assertEquals("Fahrenheit: 212, Celsius: 100.0\r\n",
26                 response.getOutputStreamContents());
27 }

```

Mocks remplace

- HttpServletRequest et
- HttpServletResponse

Création de Mock Objects

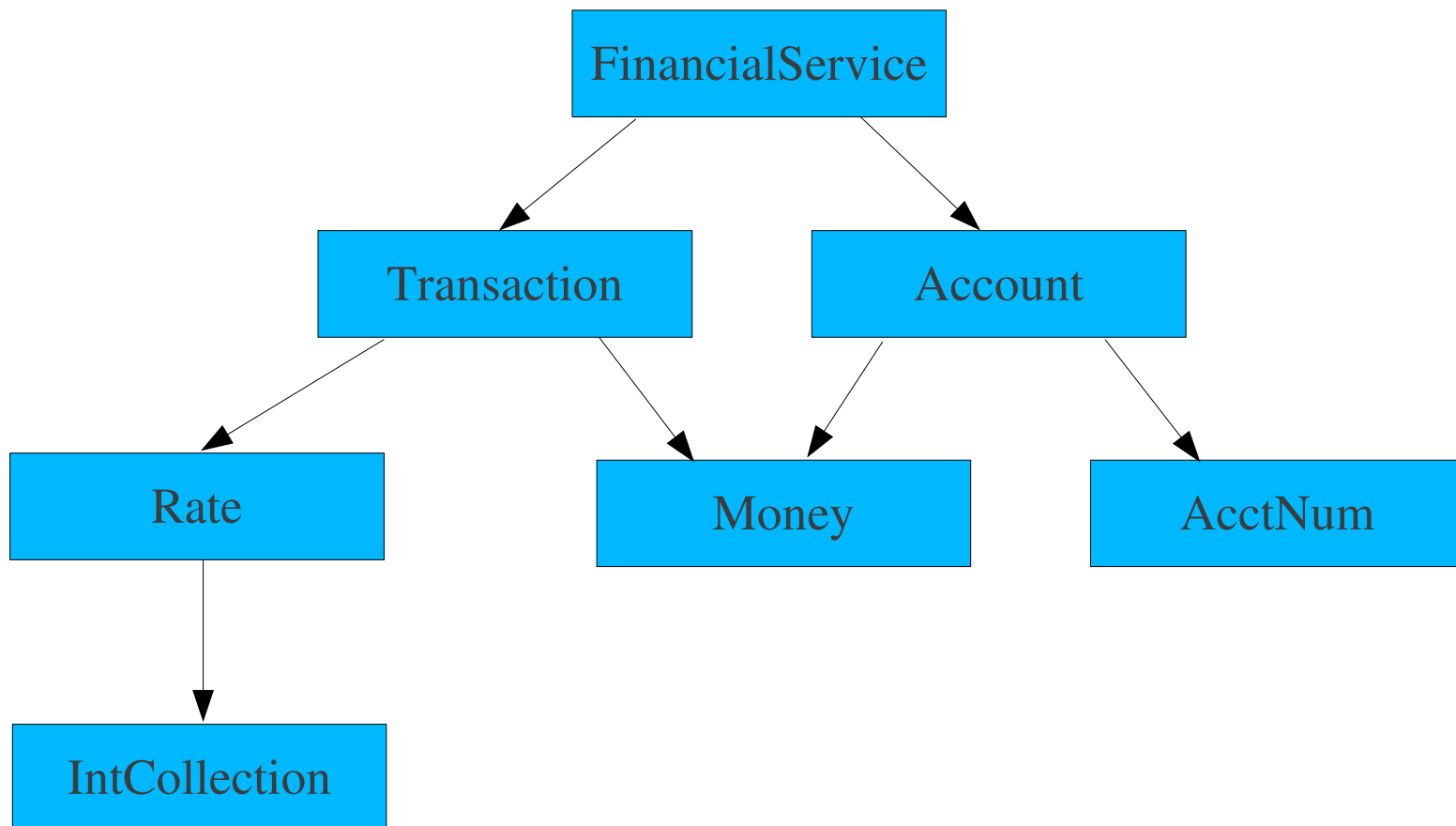
- Création manuelle
 - Avantage
 - plus de contrôle
 - Désavantage
 - peut demander la création de plusieurs classes, des difficultés avec les classes d'infrastructures existantes (ex: JDBC)
- Frameworks
 - EasyMock
 - JMock
 - MockRunner

Intégration intra-classe

- Test adéquat des opérations d'une classe (méthodes)
 - ♦ approches boîte noire, boîte blanche
 - ♦ chaque attribut mis à jour et interrogé
 - ♦ chaque exception lancée au moins une fois
 - ♦ chaque interruption forcée à intervenir au moins une fois
 - ♦ tests basés états
- Intégration de méthodes
 - Big-bang indiqué pour situation où les méthodes sont fortement couplées
 - méthodes complexes peuvent-être testés avec stubs/mocks

Intégration de Clusters

- Nécessite un arbre de dépendance de classes



Intégration de Clusters

Approche Big Bang

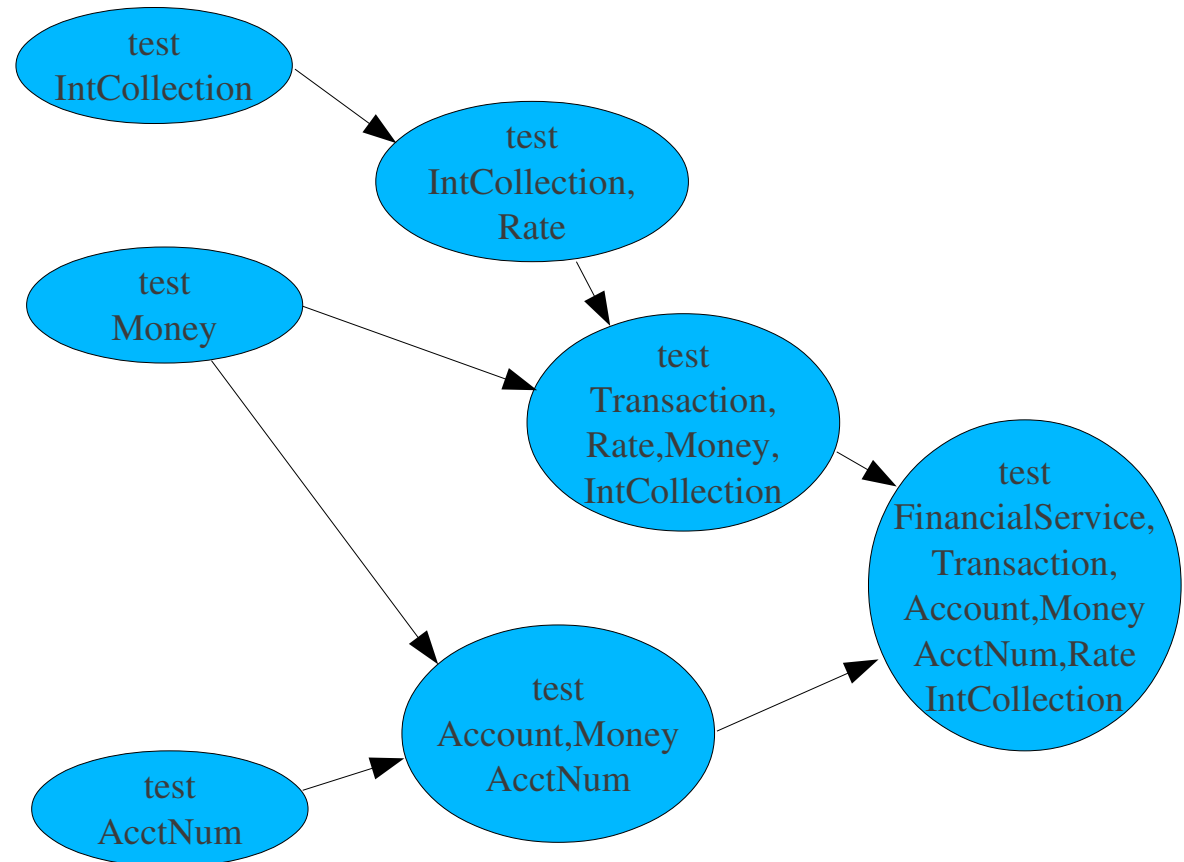
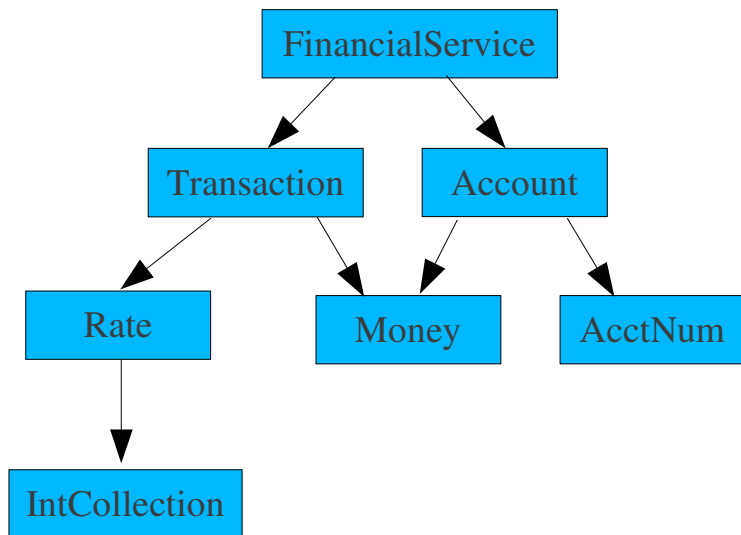
Recommandée uniquement lorsque

- Le Cluster est stable, avec peu d'éléments ajoutés/changés
- Petit cluster
- Composants sont très fortement couplés
 - impossible de tester séparément en pratique

Intégration de Clusters

Approach Bottom-up

- Technique la plus utilisée
- Intégration de classes
 - débutant aux feuilles et
 - allant vers le haut de l'arbre de dépendance



Intégration de Clusters

Intégration Top-down

- Possible uniquement lorsqu'il y a une classe de *tête*
- Exemple: cluster avec classe *facade*

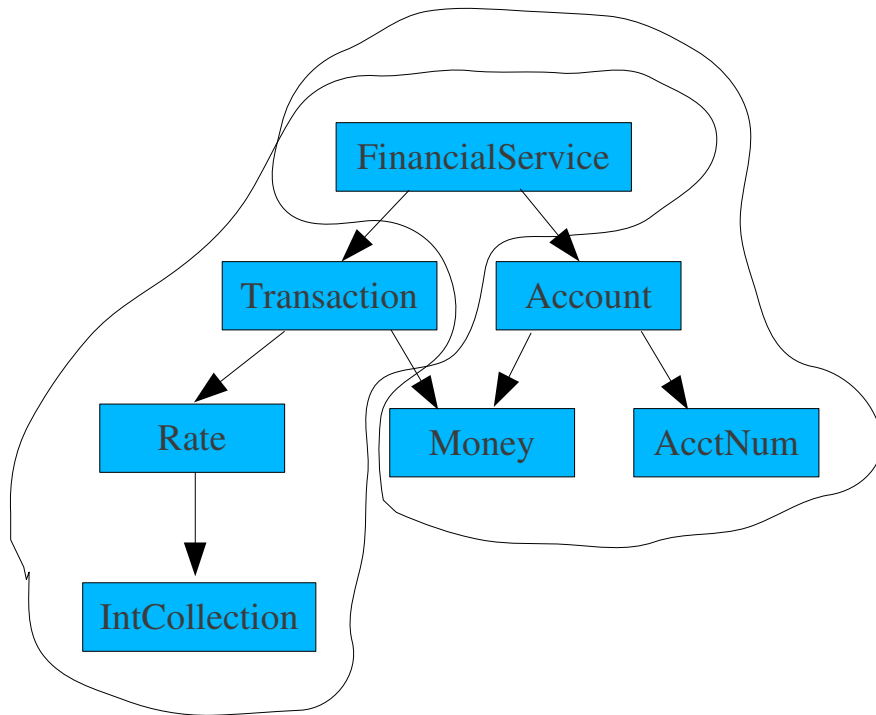
Intégration de Clusters

Approche basée Scénarios

- Scénario
 - décrit des interactions de classes
 - représentés par des diagrammes d'interactions
- Approche
 1. Lier les collaborations à l'arbre de dépendance
 2. Choisir la séquence d'application des collaborations. Ex:
 - plus simple en premier
 - avec le moins de stubs en premier
 - etc
 3. Développer et exécuter des tests selon la collaboration

Intégration de Clusters

Approche basée Scénarios



Collaboration 1

- FinancialService, Transaction, Rate, InCollection

Stubs for: Account, Money

Collaboration 2

- FinancialService, Account, Money, AcctNum

Intégration de Clusters - Problèmes liés à l'héritage

- Difficulté de compréhension du code
- Variables d'instances *similaires* aux variables globales
- Suite de test d'une méthodes quasiment jamais appropriée pour méthodes la redéfinissant (overriding)
- Ré-utilisation accidentelle
 - héritage de méthode non supposée
- Héritage multiple
 - cas de tests multiples nécessaires
 - possibilité de liaison incorrecte
 - héritage répétée
- Classes abstraites
 - besoin d'une instantiation
- Classes génériques

Exemple

```
public class Account extends Object {  
    protected Date lastTxDate, today;  
    // ...  
    int quartersSinceLastTx(){  
        return (90/daysSinceLastTx());  
    }  
    int daysSinceLastTx(){  
        return (today.day() - lastTxDay.txDate + 1);  
        // Correct - today's transaction return 1  
        //      day elapsed  
    }  
}  
  
public class TimeDepositAccount  
    extends Account{  
    //...  
    int daysSinceLastTx(){  
        return daysSinceLastTx(today.day() -  
            lastTxDay.txDate);  
        // Incorrect - today's transactions  
        //      return 0 days elapsed  
    }  
}
```

Possibilité de correctude
coincidentelle

Nécessité de tester les
méthodes héritées selon le
contexte des classes héritantes

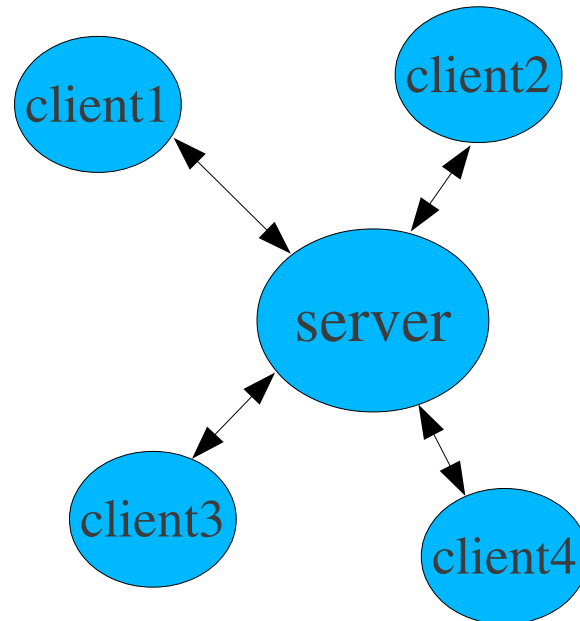
Intégration de Sous-systèmes/Systemes

- Intégration Big Bang
 - non recommandé en général
- Intégration Bottom-up
 - okay pour systèmes de petite ou taille médium
- Intégration Top-down
- Intégration basée sur Cas-d'Utilisations
 - basée sur cas d'utilisations de haut-niveau (externes)

Intégration de Clients/Serveurs

Système à deux parties (two tiers)

1. Tester chaque client avec stubs pour serveurs
2. Tester serveur avec stubs pour chaque type de client
3. Tester paires de type client avec serveur (serveur peut avoir des stubs pour les autres clients)
4. Retirer tous les stubs et tester serveur avec clients



Intégration de Clients/Serveurs

Système à trois parties (three tiers)

1. Tester chaque client avec stubs pour serveurs et middle-tier
2. Tester serveur avec stubs pour chaque type de client et middle-tier.
3. Tester chaque client avec middle-tier et proxy serveur
4. Tester serveur avec middle-tier et proxy client
5. Tester clients avec middle-tier et serveur actuel

